Universidad Nacional Autónoma de México

Facultad de Ciencias

Compiladores

Semestre 2026–2

Fecha de entrega 29/04/2026

Práctica 3

Alumno:

- Carrillo Gonzalez Luis Israel – 321050324

1.

Los símbolos no terminales son:

$$N = \{\text{programa, declaraciones, declaracion, tipo, lista_var, sentencias, sentencia, expresion}\}$$

Los símbolos terminales son:

$$\Sigma = \{\text{int, float, identificador, numero, if, else, while, ; , , , =, (,), +, -, *, /}\}$$

Símbolo inicial:

$$S = \text{programa}$$

2.

En la gramática dada sí hay ambigüedad; esta es ocasionada por el símbolo no terminal "expresión" debido a sus producciones.

Para eliminarla tenemos que eestructurar las producciones correspondientes a "expresion" introduciendo nuevos símbolos no terminales.

Las nuevas producciones serían:

- $+expresion \rightarrow +expresion + *expresion| + expresion - *expresion| * expresion$
- $*expresion \rightarrow *expresion * ()expresion| * expresion/()expresion|()expresion$
- $()expresion \rightarrow identificador|numero|(+expresion)$

3.

La gramática si tiene recursividad izquierda, la gramática sin esta es:

- $programa \rightarrow declaraciones \quad sentencias$
- $declaraciones \rightarrow declaracion \quad declaraciones'$
- $declaraciones' \rightarrow declaracion \quad declaraciones'|\epsilon$
- $declaracion \rightarrow tipo \quad lista - var \quad ;$
- $tipo \rightarrow int|float$
- $lista - var \rightarrow identificador \quad lista - var'$
- $lista - var' \rightarrow, \quad identificador \quad lista - var'|\epsilon$
- $sentencias \rightarrow sentencia \quad sentencias'$
- $sentencias' \rightarrow sentencia \quad sentencias'|\epsilon$
- $sentencia \rightarrow identificador = expresion;|$
  $if(expresion) \quad sentencias \quad else \quad sentencias|$
  $while \quad (expresion) \quad sentencias$
- $+expresion \rightarrow *expresion \quad + expresion'$
- $+expresion' \rightarrow + \quad * expresion \quad + expresion| - \quad *expresion \quad + expresion'|\epsilon$
- $*expresion \rightarrow ()expresion \quad * expresion'$
- $*expresion' \rightarrow * \quad ()expresion \quad * expresion'|/ \quad ()expresion \quad * expresion'|\epsilon$
- $()expresion \rightarrow identificador|numero|(+expresion)$

4.

Como en ninguna parte de la gramática existe un símbolo no terminal que tenga dos o más producciones que empiecen con el mismo símbolo, entonces la grámatica no se puede factorizar.

5.

$N' = \{programa, declaraciones, declaraciones', declaracion, tipo, lista_var, lista_var',$
$sentencias, sentencias', sentencia, +expresion, +expresion', *expresion, *expresion', ()expresion\}$

El conjunto P' es el siguiente:

- $programa \rightarrow declaraciones\ sentencias$
- $declaraciones \rightarrow declaracion\ declaraciones'$
- $declaraciones' \rightarrow declaracion\ declaraciones' | \epsilon$
- $declaracion \rightarrow tipo\ lista_var\ ;$
- $tipo \rightarrow int | float$
- $lista_var \rightarrow identificador\ lista_var'$
- $lista_var' \rightarrow,\ identificador\ lista_var' | \epsilon$
- $sentencias \rightarrow sentencia\ sentencias'$
- $sentencias' \rightarrow sentencia\ sentencias' | \epsilon$
- $sentencia \rightarrow identificador\ =\ +expresion\ ;$
- $sentencia \rightarrow identificador\ =\ +expresion\ ; |$
  $if\ (\ +expresion\ )\ sentencias\ else\ sentencias |$
  $while\ (\ +expresion\ )\ sentencias$
- $+expresion \rightarrow *expresion\ +expresion'$
- $+expresion' \rightarrow +\ *expresion\ +expresion' | -\ *expresion\ +expresion' | \epsilon$
- $*expresion \rightarrow ()expresion\ *expresion'$
- $*expresion' \rightarrow *\ ()expresion\ *expresion' | /\ ()expresion\ *expresion' | \epsilon$
- $()expresion \rightarrow identificador | numero | (\ +expresion\ )$

6. Conjuntos FIRST

- $FIRST(programa) = \{int, float\}$
- $FIRST(declaraciones) = \{int, float\}$
- $FIRST(declaraciones') = \{int, float, \epsilon\}$
- $FIRST(declaracion) = \{int, float\}$
- $FIRST(tipo) = \{int, float\}$
- $FIRST(lista_var) = \{identificador\}$
- $FIRST(lista_var') = \{, , \epsilon\}$
- $FIRST(sentencias) = \{identificador, if, while\}$
- $FIRST(sentencias') = \{identificador, if, while, \epsilon\}$

- $FIRST(sentencia) = \{identificador, if, while\}$
- $FIRST(+expresion) = \{identificador, numero, (\}$
- $FIRST(+expresion') = \{+, -, \epsilon\}$
- $FIRST(*expresion) = \{identificador, numero, (\}$
- $FIRST(*expresion') = \{*, /, \epsilon\}$
- $FIRST(()expresion) = \{identificador, numero, (\}$

7. Conjuntos FOLLOW

- $FOLLOW(programa) = \{\$\}$
- $FOLLOW(declaraciones) = \{identificador, if, while\}$
- $FOLLOW(declaraciones') = \{identificador, if, while\}$
- $FOLLOW(declaracion) = \{int, float, identificador, if, while\}$
- $FOLLOW(tipo) = \{identificador\}$
- $FOLLOW(lista_var) = \{;\}$
- $FOLLOW(lista_var') = \{;\}$
- $FOLLOW(sentencias) = \{\$, else, identificador, if, while\}$
- $FOLLOW(sentencias') = \{\$, else, identificador, if, while\}$
- $FOLLOW(sentencia) = \{\$, else, identificador, if, while\}$
- $FOLLOW(+expresion) = \{;,)\}$
- $FOLLOW(+expresion') = \{;,)\}$
- $FOLLOW(*expresion) = \{+, -, ;,)\}$
- $FOLLOW(*expresion') = \{+, -, ;,)\}$
- $FOLLOW(()expresion) = \{*, /, +, -, ;,)\}$

8. Tabla de análisis sintáctico predictivo

Dado que es difícil meter mucho texto dentro de una tabla, dentro de la tabla se usaron las siguientes abreviaciones:

- "programa" se abrevió como "prog."
- "declaraciones" se abrevió como "decl." y "declaracion" como "dec."
- "lista_var" se abrevió como "l_var" (y "l_v" dentro de las producciones).
- "sentencias" se abrevió como "sents." y "sentencia" como "snt."
- "+expresion" se abrevió como "+exp" (y "+e"' para "$+expresion'$ ").
- "*expresion" se abrevió como "*exp" (y "*e"' para "$*expresion'$ ").
- "()expresion" se abrevió como "()exp" (y "()e" dentro de las producciones).

Tabla de Análisis Sintáctico Predictivo $M[A, a]$ (Matriz Completa)

NT \T	int	float	id	num	if	else	while	;	,	=	(	)	+	-	*	/	$
prog.	$prog \to$ $decl\ sent$	$prog \to$ $decl\ sent$															
decl.	$decl \to$ $dec\ decl'$	$decl \to$ $dec\ decl'$															
decl.'	$decl' \to$ $dec\ decl'$	$decl' \to$ $dec\ decl'$	$decl' \to \epsilon$		$decl' \to \epsilon$		$decl' \to \epsilon$										
dec.	$dec \to$ $tipo\ l_v\ ;$	$dec \to$ $tipo\ l_v\ ;$															
tipo	$tipo \to int$	$tipo \to float$															
l_var			$l_v \to$ $id\ l_v'$														
l_var'								$l_v' \to \epsilon$	$l_v' \to$ $,\ id\ l_v'$								
sents.			$sents \to$ $snt\ snt'$		$sents \to$ $snt\ snt'$		$sents \to$ $snt\ snt'$										
sents.'			$snt' \to snt\ snt'$ $snt' \to \epsilon$		$snt' \to snt\ snt'$ $snt' \to \epsilon$	$snt' \to \epsilon$	$snt' \to snt\ snt'$ $snt' \to \epsilon$										$snt' \to \epsilon$
snt.			$snt \to$ $id = +exp;$		$snt \to if\ (+exp)$ $sent\ else\ sent$		$snt \to while$ $(+exp)\ sent$										
+exp			$+exp \to$ $*exp\ +exp'$	$+exp \to$ $*exp\ +exp'$							$+exp \to$ $*exp\ +exp'$						
+exp'								$+exp' \to \epsilon$				$+exp' \to \epsilon$	$+exp' \to$ $+\ *exp\ +e'$	$+exp' \to$ $-\ *exp\ +e'$			
***exp**			$*exp \to$ $()e\ *exp'$	$*exp \to$ $()e\ *exp'$							$*exp \to$ $()e\ *exp'$						
***exp'**								$*exp' \to \epsilon$				$*exp' \to \epsilon$	$*exp' \to \epsilon$	$*exp' \to \epsilon$	$*exp' \to$ $*\ ()e\ *e'$	$*exp' \to$ $/\ ()e\ *e'$	
()exp			$()e \to id$	$()e \to num$							$()e \to$ $(+exp)$						